

SunMint Tree-Growth Monitoring — Video Capture + Python Analysis Worker

Spec v1.3 · 2026-08-25 · Prepared by Sophia Truesight (autopilot) for Gary Teh

Scope: monitor_tree_growth module (DApp + SunMint app) — GeoJSON tree index, nearby-tree GPS selection, video capture, Python analysis worker, PM002 carbon math, on-chain event flow.

1. WHY

The SunMint model requires per-tree growth evidence that a VVB can audit. Phone-camera measurement is the proven pattern (TREEO 94–95% DBH accuracy $R^2 \geq 0.95$; ACORN/Agerpoint moving ground-truth to phone; Greenstand photo-verified). Video adds stronger tamper-evidence than a single photo — the walk-around cadence (farm-build game) is captured continuously.

2. SYSTEM ARCHITECTURE

...

Farmer phone (SunMint app + main DApp monitor page) Backend (Python worker)

- open monitor module → GPS read ■■ [ingest] receive video + metadata
- fetch GeoJSON index (cached) → rank trees ■■ [hash] SHA-256 of video → audit trail
- closest → furthest (Haversine) ■■ [process] OpenCV:
- pick tree from dropdown ■ frame extraction (e.g. 1 fps)
- record video: walk-around, calibration card visible ■ calibration-card detection (ARUCO/contour)
- auto-extract GPS + timestamp + farmer RSA signature ■ DBH estimation (pixel ratio → trig)
- save draft locally (offline-first) ■ species lookup → allometric formula
- POST signed metadata + video → /api/monitor/ingest ■ → AGB → BGB ($R=0.32$) → $\times 0.47$ → $\times 44/12$
- [emit] signed [TREE GROWTH MONITORING EVENT] → Edgar
- [notify] farmer growth result in app

...

Key decisions

- Capture + lightweight math in browser; heavy CV in Python. GAS is glue only (relay/notify), NOT the analysis engine (no OpenCV runtime).
- Hash the video before processing → reproducible, tamper-evident pipeline (VVB-grade).
- Offline-first — farmers in the field record, queue, upload when online (SunMint mobile is already offline-first).
- **Dual deployment:** the monitor page ships to BOTH the main DApp (dapp.truesight.me) AND the SunMint app (sunmint.truesight.me).

- **No database.** Tree records are served as static GeoJSON/JSON files from GitHub Pages — the DApp fetches once and filters client-side.

3. MODULE A — monitor_tree_growth (DApp + SunMint app)

3.1 Page: monitor_tree_growth.html — deployed to BOTH DApp targets

- **Main DApp:** dapp_beta/monitor_tree_growth.html → dapp.truesight.me/monitor_tree_growth.html
- **SunMint app:** truesight_me_beta/sunmint/monitor-tree-growth/index.html → sunmint.truesight.me/monitor-tree-growth/
- Same codebase/behavior in both; crypto + offline queue identical (WebCrypto RSA-2048 keypair in localStorage, IndexedDB draft queue)
- **Nearby tree selection:** GPS-based ranking closest → furthest (Haversine) → dropdown → pick tree → Start Monitoring Video
- **Tree data source:** single GeoJSON index (`sunmint/trees/index.geojson`), fetched once + client-side filter; farm-level sharding at scale; build-script generated from sheet + on-chain events
- Select farm → select/create tree record (per-tree ID, species, planting date)
- Video capture: `- Recording guidance overlay: hold card against tree at 1.3 m (breast height), portrait, walk 360° slowly
- Auto-metadata: GPS, timestamp, farmer public key
- Offline queue: IndexedDB draft storage; upload on reconnect
- Signed POST: farmer RSA signature over metadata + video hash

3.1.1 Nearby Tree Selection (UX flow)

- On opening the monitor module, the app reads the device GPS (existing geolocation permission)
- Pulls the farmer's registered trees (from farm/tree registry; each tree has lat/lng from the planting event)
- **Ranks nearby trees closest → furthest** (Haversine distance from current GPS)
- Farmer picks a tree from the dropdown → tree record selected → **Start Monitoring Video** button enables
- Fallbacks: manual search by tree ID; GPS unavailable → "All trees" list (alphabetical / last-measured first); offline → last-known GPS + cached registry
- Distance badge per tree (e.g. "12 m", "48 m", "203 m") so the farmer can walk to the nearest tree

3.1.2 GPS proximity sort (Haversine)

...

$$\text{dist} = 2R \cdot \text{asin}(\sqrt{\sin^2(\Delta\text{lat}/2) + \cos(\text{lat1}) \cdot \cos(\text{lat2}) \cdot \sin^2(\Delta\text{lng}/2)})$$
 R = 6371 km

...

- Client-side JS; pure function, unit-testable; no backend round-trip needed

3.1.3 Tree Data Source — GeoJSON index (no database)

- **Single static index:** ``sunmint/trees/index.geojson`` — a GeoJSON FeatureCollection with only `{tree_id, lat, lng, species, last_measured}`` per feature (~50–80 bytes/tree)
- 1,000 trees ≈ 80 KB · 10,000 trees ≈ 800 KB — one small download
- **Fetch once, filter client-side:** on module open the app does ``fetch('index.geojson')`` → Haversine sort in JS → nearest-first dropdown. **No server, no query params, no database.**
- **Caching:** localStorage / service worker with 24–48h TTL → repeat visits cost nothing; offline works from the cached copy
- **Scale rule — shard by farm, not by database:** when the index grows, split into ``trees/by-farm/<farm_id>.geojson`` + a tiny ``trees/farms.json`` map; the DApp fetches the farm map → fetches only farm files within ~10 km of the current GPS. Same static-file pattern, more files — never a server-side query.
- **Build rule (machine-owned):** a build script regenerates the GeoJSON files from the Google Sheet (``SunMint Tree Planting`` tab) + on-chain events — same pattern as treasury-cache / contributors-cache. The sheet + on-chain ledger stay authoritative; the GeoJSON is a **derived, versioned cache** on GitHub Pages (git history = VVB-visible provenance).
- **Full tree records:** each tree's complete object (with measurement history) lives at ``sunmint/trees/<tree_id>.json`` — fetched only when the farmer selects that tree (not in the index). Measurements append to the tree's own file via the Python worker.

3.2 Data contract (request)

```
```json
{
 "farmer_id": "<did/farm>",
 "tree_id": "SUNMINT-0001",
 "species": "cacao | brazil_nut | acai | mahogany | jatoba",
 "video_b64": "<base64 or multipart>",
 "video_sha256": "<hex>",
 "gps": {"lat": -14.5, "lng": -39.0},
 "captured_at": "2026-08-25T14:30:00Z",
 "farmer_signature": "<rsa-sig over sha256 of metadata+video_hash>",
 "nearby_trees": [{"tree_id": "SUNMINT-0001", "distance_m": 12}] // context: sorted closest→furthest at capture
}
```
```

3.3 Data contract (response)

```
```
{ status, measurement_id, dbh_cm, height_m, agb_kg, co2e_kg,
```

```
confidence, analysis_sha256, edgar_event }
...
```

## 4. MODULE B — Python analysis worker (truesight\_autopilot)

### 4.1 Stack

- Python 3.11+, OpenCV (frame extraction, ARUCO card detection), numpy, hashlib
- Optional: scikit-image / lightweight YOLO for tree segmentation (v2 polish)
- Crypto: farmer public key verification (RSA-2048), result signing (Sophia DAO key)
- Deployment: on the autopilot box (or a small EC2 worker) exposed as /api/monitor/ingest

### 4.2 Pipeline (pure functions, unit-testable)

1. `extract_frames(video, fps=1) → frames[]`
2. `detect_calibration_card(frame) → card corners (ARUCO marker or contour; ISO 7810 85.6×53.98 mm)`
3. `estimate_dbh(frame, card_bounds, px_to_mm) → DBH at 1.3 m (similar-triangle / pixel-ratio)`
4. `allometric_agb(species, dbh) → AGB kg (species table; IPCC/Chave allometrics)`
5. `carbon_math(agb) → PM002 chain:  $\Delta BGB = \Delta AGB \times R$  ( $R=0.32$ ) ·  $CO_2e = ((\Delta AGB + \Delta BGB) \times 0.47) \times 44/12 \times (1-A_{pre})(1-A_{unc})(1-LD)(1-AR)(1-RB) - (E_{proj} - E_{base}) \cdot AR=10\% \cdot RB=20\%$`
6. `sign_result(measurement) → measurement_id + signature`
7. `emit_event(result) → [TREE GROWTH MONITORING EVENT] → Edgar`

### 4.3 Allometric species table (v1 seeds; needs field calibration)

Species	Formula source	Notes
Cacao (Theobroma cacao)	Chave et al. tropical / local Bahia equations	needs validation
Brazil nut	Chave tropical moist	—
Acai	palm-specific AGB	—
Mahogany	Chave tropical	—
Jatoba	Chave tropical	—

### 4.4 Measurement append (to tree JSON)

- After analysis, the worker appends the measurement to ``sunmint/trees/<tree_id>.json`` (measurement history)
- Index entry updated: `last_measured` + latest `co2e` → regenerated by build script

## 5. EVENT TYPES

- [TREE GROWTH MONITORING EVENT] — new event; per-measurement on-chain record: tree\_id, dbh, agb, co2e, gps, captured\_at, analysis\_sha256, farmer\_sig, worker\_sig
- Reuses existing [TREE PLANTING EVENT] for the initial planting record
- Plan Vivo / VVB audit path: raw video (stored) + hash + analysis log = full provenance

## 6. IMPLEMENTATION ROADMAP (phased)

Phase	Deliverable	Effort	Owner	Exit criteria
P0 — Spec & contract	This spec; API contract frozen	0.5 d	Sophia	Gary sign-off
P1 — Browser capture	monitor_tree_growth.html in dapp_beta AND sunmint/monitor-tree-growth in truesight_me_beta: <b>GeoJSON index + nearby-tree GPS ranking (Haversine) + dropdown</b> , video capture, offline queue, signed POST	5–6 d	Dev (beta-first)	UAT on both beta sites
P2 — Python worker MVP	ingest → frames → card detect → DBH → CO <sub>2</sub> e → signed result	3–5 d	Sophia/Dev	Unit tests pass (compileall, ruff, pytest)
P3 — Integration	worker endpoint ↔ dapp POST; response contract	2 d	Dev	End-to-end on beta
P4 — On-chain event	[TREE GROWTH MONITORING EVENT] in dao_protocol/Edgar	2 d	Dev	Event lands on ledger
P5 — Growth history view	per-tree measurements over time; farmer dashboard	2–3 d	Dev	History render on beta
P6 — Field calibration	species allometrics vs tape/calipers on pilot plots	ongoing	Field team	$R^2 \geq 0.9$
P7 — VVB audit prep	analysis reproducibility doc + video retention	1–2 d	Sophia	VVB-ready evidence pack

## 7. CHECKLIST (execution tracking)

- [ ] P0: spec signed off by Gary

- [ ] P1: monitor\_tree\_growth.html drafted (dapp\_beta PR)
- [ ] P1: sunmint/monitor-tree-growth/index.html drafted (truesight\_me\_beta PR) — sunmint.truesight.me target
- [ ] P1: GeoJSON index generated (build script from sheet + on-chain)
- [ ] P1: nearby-tree GPS ranking (Haversine) + dropdown implemented
- [ ] P1: offline queue (IndexedDB) implemented
- [ ] P1: UAT on beta sites
- [ ] P2: worker scaffold in truesight\_autopilot
- [ ] P2: extract\_frames + detect\_calibration\_card
- [ ] P2: estimate\_dbh + allometric\_agb + carbon\_math (PM002)
- [ ] P2: sign\_result + emit\_event
- [ ] P2: LOCAL TEST SUITE PASSES (compileall, ruff, ruff format, pytest)
- [ ] P3: endpoint wired to DApp POST
- [ ] P4: Edgar event type registered
- [ ] P5: growth history view on beta
- [ ] P6: field calibration data collected
- [ ] P7: VVB evidence pack assembled

## 8. RISKS & OPEN QUESTIONS

- Card detection in harsh light / occlusion → ARUCO markers more robust than plain contours; fallback to manual DBH entry + photo evidence (TREEO-style)
- Species allometrics for cacao are weak → P6 field calibration is critical; Plan Vivo allows project-specific equations
- Video size (50–200 MB) → compress to  $\leq 720p$  / 10 fps in app before upload; worker cap 5 min video
- VVB acceptance of phone-video MRV → Plan Vivo participatory monitoring precedent (CommuniTree since 2010); keep raw video + hashes
- GPS accuracy ( $\pm 5$ –10 m) may mis-rank trees on dense plots → distance badges help; nearest-3 shown with equal priority
- GeoJSON staleness → build script TTL (24–48h) + on-open refresh; stale-tolerant UI (badge "data age")
- Worker deployment → autopilot box first; scale to dedicated worker if volume demands